

INFORME DEL PROYECTO: ASISTENTE DE MENÚS SALUDABLES

índice

1. Descripción del proyecto..... 2

2. URL del repositorio.....2

 ➤ Repositorio → Projecte_Digitalitzacio..... 2

 ➤ Historial Commit..... 2

3. Uso de la IA..... 3

 ➤ Herramienta y modelos utilizados..... 3

 ➤ Método de integración de Gemini..... 3

4. Flujos de trabajo.....4

 ➤ Pasos del desarrollo:..... 4

 → Prompting..... 4

 → Enfoque agéntico..... 4

 → Automatización..... 5

5. Explicación del código..... 5

 ➤ Configuración de la página..... 5

 ➤ Estilos y diseño..... 5

 ➤ Herramienta auxiliares.....6

 ➤ Barra lateral (panel de opciones).....7

 ➤ Parte principal (carga de imagen)..... 8

 ➤ Análisis y resultados..... 9

 ➤ Pie de página..... 10

6. Tecnologías utilizadas..... 11

 ➤ Gemini CLI /API..... 11

 ➤ Node.js..... 11

 ➤ Python 3.13.....11

 ➤ Streamlit..... 11

 ➤ Pillow (PIL)..... 11

 ➤ Git / GitHub..... 12

7. Retos y aprendizajes..... 12

 ➤ Dificultades encontradas..... 12

 → Elegir la versión / modelo correcta de la IA..... 12

 → Definir reglas para las respuestas.....12

Liu Qi Zhou

Curso: 2025-2026

Asignatura: Digitalización aplicada a los sectores de productivos

Grupo: IFC31A

8. Limitaciones y mejoras futuras.....	13
➤ Limitaciones.....	13
➤ Mejoras futuras.....	13
9. Reflexión sobre el uso de la IA.....	13
➤ Qué partes han funcionado bien.....	13
➤ Qué partes han decepcionado.....	13
➤ Cómo se ha calibrado las respuestas.....	14
10. Instrucciones de instalación y uso.....	14
Windows.....	14
Linux.....	14

Liu Qi Zhou

Curso: 2025-2026

Asignatura: Digitalización aplicada a los sectores de productivos

Grupo: IFC31A

1. Descripción del proyecto

Este proyecto consiste en una aplicación web interactiva, diseñada para analizar platos de comida o menús completos a partir de las fotografías proporcionadas por el usuario.

Su objetivo principal es utilizar la Inteligencia Artificial para evaluar si los alimentos son saludables, indicar sus propiedades nutricionales, detectar posibles alérgenos y ofrecer recomendaciones útiles para mejorar los hábitos de alimentación.

Con la ayuda integrada de la IA, permite resolver la necesidad de obtener toda la información detallada de forma rápida y visual. Todo el sistema está programado en lenguaje Python y funciona mediante la librería Streamlit, que permite transformar el código en una interfaz web visual y fácil de usar.

2. URL del repositorio

➤ Repositorio → Projecte_Digitalitzacio

- https://github.com/lzhou190/Projecte_Digitalitzacio/tree/Projecte_Digitalitzaci%C3%B3

➤ Historial Commit

- https://github.com/lzhou190/Projecte_Digitalitzacio/commits/Projecte_Digitalitzaci%C3%B3/

3. Uso de la IA

➤ Herramienta y modelos utilizados

Para el procesamiento de imágenes y generación de respuestas se ha utilizado el servicio Google Gemini, eligiendo la versión **gemini-3-flash-preview** como la versión final, por su escalabilidad, rendimiento y capacidad de uso gratuito (encontrado en la página oficial, sin limitaciones).

Otras versiones utilizados para la prueba:

- **Gemini 1.5 / latest:** no reconoce la clave API nueva, deja de responder / errores de compatibilidad.
- **Gemini 2.0 / latest:** más potente, pero tiene limitaciones estrictos de uso de la cuota.
- **Gemini 2.0 Lite:** versión ligera, pero no analiza bien los alimentos y responde con información incompleta, a veces con limitaciones de cuota.

Esta herramienta realiza las funciones principales del sistema:

- Identificar los alimentos y platos en la imagen.
- Estimar valores nutricionales y detectar posibles alérgenos.
- Generar recomendaciones y adaptar las respuestas al idioma seleccionado por el usuario.

➤ Método de integración de Gemini

Está conectado mediante la librería *google-generativeai* para python.

El sistema recibe la clave introducida por el usuario, configura el modelo y prepara la imagen, cuando recibe la foto del usuario la organiza y muestra los resultados.

También incluye el modo Demo, este funciona sin la necesidad de claves, generando resultados con ejemplos guardados al código.

4. Flujos de trabajo

➤ Pasos del desarrollo:

- **Estructura inicial:** configurar la página para que ocupa toda la pantalla, definir el título + ícono; Organizar la interfaz en barra lateral y en la zona principal con diseño visual básico.
- **Panel de control:** crear las opciones para activar y desactivar el modo Demo, el campo para introducir clave API y selección de idioma.
- **Definir el funcionamiento:** 2 modos, el Demo (mostrar resultados con ejemplos guardados) y el modo real (lee la clave y conecta con el servicio analiza con resultados inteligentes).
- **Elección del modelo Gemini:** conectar al servicio en la página web para probar la función de IA, descartando aquellos modelos incompatibles / con restricciones.
- **Ajuste de las instrucciones:** realizar pruebas enviado mensajes a la IA, obligando que empiece con una calificación y luego análisis de 4 campos diferentes.
- **Procesamiento de datos:** programar la lectura automática de la calificación asignando un color diferente según el nivel nutricional.
- **Verificación final:** después de conseguir todos los códigos, comprobar que todo va bien.

➔ Prompting

Redactar instrucciones claras y detalladas, indicando qué analizar, en qué idioma y todos los pasos necesarios a seguir.

➔ Enfoque agéntico

Diseñar el sistema para que funcione por sí mismo: recibir la imagen, gestionar la conexión, obtener el resultado organizando automáticamente.

Liu Qi Zhou

Curso: 2025-2026

Asignatura: Digitalización aplicada a los sectores de productivos

Grupo: IFC31A

→ Automatización

Programar reglas para que el sistema asigne iconos o cambie el color según el nivel nutricional, y notifique los errores por sí mismo.

5. Explicación del código

➤ Configuración de la página

Importa las librerías necesarias para funcionar: Streamlit (facilidad de desarrollo con códigos simples), google.generativeai, time (controla el tiempo y pausas en el programa), re (busca, lee y modifica partes de textos con reglas). Luego se configura la ventana del navegador con título, icono adaptando el ancho para aprovechar todo el espacio de la pantalla.

```
import streamlit as st
from PIL import Image
import google.generativeai as genai
import time
import re

st.set_page_config(page_title="Menú Saludable IA", page_icon="🍌",
layout="wide")
```

➤ Estilos y diseño

Se define la apariencia: colores de fondo degradados, cajas con efecto transparente, bordes redondeados y animación suaves. También se crean reglas de colores para que la nota de calidad nutricional se vea en verde, amarillo, rojo automáticamente.

Se usa CSS para lograr un diseño atractivo, se crean clases reutilizables para mantener todo con el mismo estilo.

```
.stApp {
    background: linear-gradient(135deg, #FF0076 0%, #590FB7 45%,
#FFD600 100%);
    background-attachment: fixed;
}
```

Liu Qi Zhou

Curso: 2025-2026

Asignatura: Digitalización aplicada a los sectores de productivos

Grupo: IFC31A

```
/* Contenedores Glassmorphism con tamaño responsivo */
.info-header-container {
  background: rgba(0, 0, 0, 0.25);
  backdrop-filter: blur(15px);
  padding: clamp(15px, 3vw, 30px);
  border-radius: 35px;
  border: 3px solid #FF0076;
  box-shadow: 0 0 25px rgba(255, 0, 118, 0.4);
}
```

```
/* Botón ANALIZAR (Responsivo) */
.stButton>button {
  border-radius: 25px;
  background: linear-gradient(90deg, #FF512F 0%, #DD2476 100%);
  transition: all 0.3s cubic-bezier(0.175, 0.885, 0.32, 1.275);
}
```

Calidad nutricional:

```
.score-A { background-color: #008b4c; }
.score-B { background-color: #85bb2f; }
.score-C { background-color: #fecb02; color: #000; }
.score-D { background-color: #ee8100; }
.score-E { background-color: #e63e11; }
```

➤ Herramienta auxiliares

Incorporación de 2 herramientas importantes, la primera lee el texto y pone iconos automáticos adaptando la palabra. La segunda transforma el texto plano que envía a IA en un formato web limpio y con listas ordenadas.

```
# FUNCIONES DE UTILIDAD PARA ICONOS DINÁMICOS
def get_dynamic_icon(content, default_icon):
    content = content.lower()
    # Mapeo de palabras clave a iconos
    mapping = {
        "proteína": "🍗", "pollo": "🐔", "carne": "🥩", "pescado":
"🐟", "huevo": "🥚"
```

Liu Qi Zhou

Curso: 2025-2026

Asignatura: Digitalización aplicada a los sectores de productivos

Grupo: IFC31A

```

}

for key, icon in mapping.items():
    if key in content:
        return icon
    return default_icon

def parse_markdown_to_html(text, default_bullet="✦"):
    # Convertir negritas
    text = re.sub(r'\*\*(.*?)\*\*', r'<b>1</b>', text)

```

➤ Barra lateral (panel de opciones)

Aquí sitúa los códigos del menú de ajustes, se añade un interruptor para activar (permite análisis sin clave pero con respuesta por defecto) y desactivar el modo Demo para usar la IA real, demostrando un campo para escribir la clave API, además dispone un campo para escribir el idioma que desea el usuario.

Por la seguridad de la clave, ha especificado que la contraseña debe ser *type = "password"*.

```

# 3. BARRA LATERAL
with st.sidebar:
    st.image("https://cdn-icons-png.flaticon.com/512/2424/2424569.png",
width=80)
    st.markdown("<h1 style='color: white; text-align: center;'><span
class='sidebar-icon'>🚀</span> Configuración</h1>",
unsafe_allow_html=True)
    st.divider()
    demo_mode = st.toggle("🚀 Activar Modo Demo", value=False)
    if demo_mode:
        tipus_demo = st.selectbox("Simulación:", ["Menú Complet", "Plat
Únic (Recepta)"])
    else:
        api_key = st.text_input("Gemini API Key:", type="password")

```


➤ Parte principal (carga de imagen)

El siguiente código organiza la zona central en 2 columna:

A la izquierda dispone el botón para seleccionar la foto, y si no hay ninguna se muestra un aviso.

A la derecha, si se cargó correctamente la imagen, se muestra la vista previa (en contrario aparece un recuadro de espera). Finalmente debajo de las columnas sitúa un botón al centro de la página.

```
# 5. CUERPO DE LA APLICACIÓN
col_left, col_right = st.columns(2, gap="medium")
with col_left:
    uploaded_file = st.file_uploader("", type=["jpg", "jpeg", "png"],
label_visibility="collapsed")
    if not uploaded_file:
        st.info("👉 Selecciona una foto desde tu dispositivo")

with col_right:
    if uploaded_file:
        image = Image.open(uploaded_file)
        st.image(image, use_container_width=True, caption="Imagen
cargada correctamente")
    else:
        st.markdown("""
            <div style="border: 3px dashed rgba(255,255,255,0.2);
border-radius: 20px; height: 150px; display: flex; align-items: center;
justify-content: center; color: rgba(255,255,255,0.4); font-weight:
bold;">

                🖼 Esperando imagen...

            </div>
            """, unsafe_allow_html=True)

st.markdown("<br>", unsafe_allow_html=True)
_, col_btn, _ = st.columns([1, 1.2, 1])
with col_btn:
    analizar = st.button("🔍 ANALIZAR AHORA")
```

➤ Análisis y resultados

Esta parte adapta reglas estrictas:

Solo se activa si hay imagen cargada y el usuario ha pulsado el botón; En modo Demo se carga resultados de ejemplo con una nota de calidad. Si está en modo real, verifica la existencia de la clave y conecta con Gemini, con el idioma elegido por el usuario, la IA genera respuestas extrayendo la calificación nutritiva, pero si falla la clave / error muestra un mensaje informativo.

```
# 6. RESULTADOS UNIFICADOS
if uploaded_file and analitzar:
    st.markdown("<hr>", unsafe_allow_html=True)
    st.markdown("<div class='info-header' style='text-align:
center;'>📊 RESULTADO DEL ANÁLISIS</div>", unsafe_allow_html=True)

    with st.spinner('⚙️ Analizando nutrición...'):
        time.sleep(1.5)

        html_output = "<div class='result-card'>"

        if demo_mode:
            html_output += "<div class='nutri-score-inline
score-A'>Calidad Nutricional: A - Excelente</div>"
            demo_data = [
                ("Identificación", "🥗", "😊", "* Ensalada Saludable\n*
Agua mineral\n* Pechuga de pollo")...
            ]
        else:
            try:
                if not api_key:
                    st.warning("⚠️ Introduce tu API Key en la barra
lateral.")

                html_output = ""
            else:
```

```
genai.configure(api_key=api_key)
model =
genai.GenerativeModel('gemini-3-flash-preview')

prompt = f"""
Analiza esta imagen nutricionalmente. Responde en
el idioma: {idioma_analisis}.

response = model.generate_content([prompt, image])
raw_text = response.text

# Extraer Nutri-Score
score_match = re.search(r'\[SCORE: ([A-E])\]',
raw_text)

if score_match:
    score = score_match.group(1)
    labels = {"A": "Excelente", "B": "Buena", "C":
"Intermedia", "D": "Baja", "E": "Desfavorable"}
    html_output += f"<div class='nutri-score-inline
score-{score}'>Calidad Nutricional: {score} - {labels[score]}</div>"
except Exception as e:
    st.error(f"❌ Error: {e}")
```

➤ Pie de página

Esta parte muestra el final de la página con el nombre del proyecto (cierre visual).

```
# 7. PIE DE PÁGINA

st.divider()

st.markdown("<div class='custom-footer'>🚀 PROYECTO DIGITALIZACIÓN 2026
| DESARROLLADO CON GEMINI AI</div>", unsafe_allow_html=True)
```

Liu Qi Zhou

Curso: 2025-2026

Asignatura: Digitalización aplicada a los sectores de productivos

Grupo: IFC31A

6. Tecnologías utilizadas

➤ Gemini CLI /API

Motor de inteligencia artificial que hace funcional todo el sistema. Se utiliza desde la terminal Powershell mediante el siguiente comando:

npx.cmd @google/gemini-cli

➤ Node.js

Entorno necesario que debe estar instalado previamente, ya que aporta herramientas para ejecutar los comandos relacionados con la interfaz de línea de comandos de Gemini.

➤ Python 3.13

Lenguaje de programación usado para todo el código de la aplicación, la gestión de imágenes y la conexión con la IA.

➤ Streamlit

Biblioteca de Python encargada de transformar el código en una página web moderna. Es la responsable de la página web (estilos, botones, navegación visualmente). Sin necesidad de diseñar páginas webs complejas.

➤ Pillow (PIL)

Biblioteca utilizada para el tratamiento, lectura y carga de las imágenes que sube el usuario, adaptándolas al formato necesario antes de ser procesadas.

Liu Qi Zhou

Curso: 2025-2026

Asignatura: Digitalización aplicada a los sectores de productivos

Grupo: IFC31A

➤ **Git / GitHub**

Sistema que ha utilizado mientras realiza el proyecto, guarda cada cambio que se hace, permite volver a versiones anteriores si algo falla, manteniendo todo el código ordenado y seguro.

7. Retos y aprendizajes

➤ **Dificultades encontradas**

➔ **Elegir la versión / modelo correcta de la IA**

Al inicio del desarrollo, había probado las versiones antiguas del modelo Gemini, pero resultaron incompatibles con las claves de acceso actuales, algunas presentaban fallos o errores (401, 404), algunos muestran el fallo por superar la cuota.

- Solución: tenía que realizar múltiples pruebas hasta identificar la versión ***gemini-3-flash-preview***.
- Aprendizaje: se ha comprendido la importancia de elegir la versión correcta de las herramientas y cómo actualizar el desarrollo ante los cambios de las versiones.

➔ **Definir reglas para las respuestas**

Inicialmente, ante la misma imagen, la inteligencia artificial generaba respuestas con estructuras, textos u órdenes diferentes que dificulta la lectura de los resultados.

- Solución: se redactaron instrucciones muy detalladas y estrictas, definiendo exactamente qué información debía aparecer, en qué orden y con qué formato.
- Aprendizaje: se ha aprendido a diseñar indicaciones precisas para guiar el comportamiento de la IA, hasta obtener un resultado ajustado a lo requerido.

8. Limitaciones y mejoras futuras

➤ Limitaciones

- No ha podido calcular los valores nutritivos para que sean 100% exactos.
- No se puede garantizar que funcione bien si las imágenes no se ven bien (borrosas), por eso el resultado del análisis puede ser distinto.
- A veces la IA cuesta reconocer los nombres de los menús / platos, por eso a la hora de traducir, los resultados pueden ser un poco distintos.

➤ Mejoras futuras

- Añadir una parte dónde el usuario pueda ver las historiales de los análisis.
- Crear una columna dónde permitan comparar diferentes fotos a la vez. Así el usuario puede decidir qué escoger.

9. Reflexión sobre el uso de la IA

➤ Qué partes han funcionado bien

Ha funcionado bien la capacidad para analizar las imágenes y entender qué tipo de comidas hay en la foto. La IA responde de manera rápida y crea textos claros con los datos principales.

También funciona bien la posibilidad de cambiar el idioma al orden de la información cuando se le pide.

➤ Qué partes han decepcionado

Ha decepcionado ver que los resultados de los valores nutritivos no sean exactos, a veces traducen las palabras (especialmente nombres de la comida) de forma extraña.

➤ Cómo se ha calibrado las respuestas

Para mejorar los resultados del análisis, se han probado varias veces con la misma imagen, y corrigiendo las instrucciones paso por paso hasta conseguir que la información sea lo más correcta posible.

10. Instrucciones de instalación y uso

Antes de ejecutar la aplicación, se debe tener instalados las siguientes herramientas.

- **Node.js**

Windows

- Windows 64-bit Installer:
<https://nodejs.org/dist/v26.1.0/node-v26.1.0-x64.msi>
- Windows ARM 64-bit Installer:
<https://nodejs.org/dist/v26.1.0/node-v26.1.0-arm64.msi>
- Windows 64-bit Binary: <https://nodejs.org/dist/v26.1.0/win-x64/node.exe>
- Windows ARM 64-bit Binary:
<https://nodejs.org/dist/v26.1.0/win-arm64/node.exe>

Linux

- Linux 64-bit Binary:
<https://nodejs.org/dist/v26.1.0/node-v26.1.0-linux-x64.tar.xz>
 - Linux PPC LE 64-bit Binary:
<https://nodejs.org/dist/v26.1.0/node-v26.1.0-linux-ppc64le.tar.xz>
 - Linux s390x 64-bit Binary:
<https://nodejs.org/dist/v26.1.0/node-v26.1.0-linux-s390x.tar.xz>
- **Python 3.13**
 - Clave de acceso API: se consigue en la página oficial de Google AI Studio de forma gratuita, pero se debe iniciar la cuenta de Google.

Liu Qi Zhou

Curso: 2025-2026

Asignatura: Digitalización aplicada a los sectores de productivos

Grupo: IFC31A

A continuación instala la herramienta Streamlit, abre Powershell y situar en la ruta donde se guarda la aplicación con extensión .py, escriba el siguiente comando:

py -m pip install streamlit google-generativeai Pillow

Poner en marcha la aplicación:

py -m -streamliit run "nombre_app.py"